

SYSTEM_MAP.md — Timhirt School SaaS: Super-Admin Tenant Provisioning & Full System Audit

Phase: 0 — Recon only (no writes performed) **Repo path:** `/home/user/timhirt-school-saas` **Run date:** 2026-08-10 **Auditor:** Claude Code (super-admin/QA-auditor role, per the source prompt)

Scope note — read this before anything else

The source prompt's own final instruction is: *"Begin with Phase 0 only. Print the system map summary and your Phase 1 onboarding plan, then stop and wait for my approval."* This document honors that literally. Nothing below reflects Phase 1–6 execution — no school was onboarded, no login was attempted, no isolation test was run. Every claim here is grounded in the repository's own source and migration files, cited `file:line`, exactly as Hard Rule item 30 of the source prompt requires ("cite file:line for every claim").

Phase 1 is currently blocked — see §Z. This is itself the first real finding: the repo has no configured non-production Supabase project to safely test against.

1. Repo structure & routing tree

React + Vite + TanStack Query frontend, Supabase (Postgres + RLS + Edge Functions + Storage) backend, no custom API server. `src/app/router.tsx` (418 lines) is the single routing tree.

Role-gate chain, applied per top-level route group:

```
RequireAuth → RequireRole roles={[...] } → RequireModule module="..."
```

- `src/features/auth/RequireRole.tsx:1-16` — redirects to `/` unless `profile.role` is in the route's allowed list. `super_admin` explicitly bypasses every `roles` check (line 14), matching the same bypass clause every backend RLS policy carries. The file's own comment documents *why*: a prior deployment had `super_admin` accounts unable to navigate anywhere, because `"super_admin"` deliberately never appears in any route's tenant-facing `roles` list (those lists describe school-side roles; the super-admin surface is `/platform/*`).
- `src/features/auth/RequireModule.tsx:1-15` — gates a route behind the tenant's enabled-module set (`useEnabledModules`). Also bypasses for `super_admin`.

Confirmed gap (not yet live-verified): `RequireModule.tsx:2-4`'s own comment states module gating is *"UX-only... RLS/DB enforcement of module gating is a deliberate follow-up, not done here."* That means a tenant whose plan/`tenant_module_overrides` disables a module (e.g. `hostel`, `library`) is only kept out of it by the React router — a direct PostgreSQL call or a guessed URL for a disabled module's tables is **not** blocked by

RLS on the module-gate dimension (per-table tenant/role RLS still applies normally; it's specifically the *module toggle* that isn't enforced server-side). This is a finding for the Phase 3 walk (Platform module), not fixed here.

Route groups observed in `router.tsx` (role lists abbreviated as they appear in the constants used, e.g. `ADMIN_REG`, `FINANCE`, `LIBRARY`, `TEACH`): academic/admissions/ID cards (`ADMIN_REG`, lines 148–166), school-admin-only fees/communication/hostel/discipline/clinic/transport/events (line 176+), finance (`FINANCE`, line 235), library (`LIBRARY`, line 263), teaching (`TEACH`, line 278). Full enumeration of every route × role was not transcribed line-by-line in this pass — recommend a scripted diff of `router.tsx` against the resource-permission catalog (§3) before Phase 3, so "reachable in the nav" and "actually authorized" can be compared mechanically rather than by eye.

2. Tenancy model

`supabase/migrations/20260713000001_core.sql`:

- `tenants` (line 17) — `id`, `name` (2–120 chars, checked), `slug` (regex-checked), `status` enum (`active|suspended|trial`).
- `users` (line 26) — profile mirror of `auth.users`; `tenant_id` nullable **only** for `super_admin`, enforced by a table CHECK constraint (`tenant_required_unless_super`, line 35-36), not just convention.
- **Security-definer resolution helpers** (lines 41-49), the single source of truth every RLS policy calls rather than trusting a client-supplied claim: `sql get_tenant_id_for_user(user_id uuid) → select tenant_id from users where id = user_id` `get_role_for_user(user_id uuid) → select role::text from users where id = user_id` Both are `security definer`, `stable`, `search_path = public`. Because every RLS policy calls these instead of reading a JWT claim directly, a forged `tenant_id` in a request body or URL has no effect on authorization — the source of truth is always the `users` row for the *authenticated* `auth.uid()`, which the client cannot rewrite. This directly addresses the source prompt's stated regression-check concern (§8 item 1: "no policy relying solely on a client-supplied tenant_id") — **preliminary verdict: architecturally sound**, pending the live negative tests in Phase 2, which have not been run.
- `user_role` enum (`20260713000001_core.sql:10-12`) originally 8 values: `super_admin`, `school_admin`, `teacher`, `student`, `parent`, `hr_officer`, `accountant`, `registrar`. Two values were added later via `ALTER TYPE ... ADD VALUE`:
- `librarian` — `supabase/migrations/20260813000001_library_role_and_enum.sql:10`
- `pending` — `supabase/migrations/20260817000008_sso_pending_role_enum.sql:8` (used for JIT-provisioned SSO users awaiting activation — relevant to the separate SAML SSO check-in tracked outside this audit).

So the live role set is 10 values, not the 8 named in the source prompt's §2 role matrix list; the prompt's list is otherwise a subset match (it doesn't separately name `librarian`, `pending`, or a `cashier` role — "accountant/cashier" in the prompt appears to be describing the `accountant` role's real-world job title, not a second enum value; no distinct `cashier` role exists in code).

3. Role matrix as actually enforced

Two layers, evolved over the project's history:

1. **Coarse RLS role checks** — the majority of policies (§4) gate `insert /update/delete` by `get_role_for_user(auth.uid()) in (...)` directly in the policy body, e.g. `students_write` (`20260713000005_rls_policies.sql:80-84`) restricts writes to `school_admin, registrar`.
2. **Fine-grained resource-permission system** (newer, layered on top for settings-driven per-role/per-user control):
 - `builtin_role_permission_grants` —
 - `20260816000001_resource_permissions.sql:65` - `user_permission_overrides` —
 - `20260816000001_resource_permissions.sql:76` - `resource_open_actions` /
 - `resource_default_role_grants` — introduced in a rewrite,
 - `20260817000001_resource_permissions_core_v2.sql:49-60` -
 - `has_resource_permission(p_user_id, p_resource, p_action)` — resolution function, **rewritten four times** as the system hardened: `20260816000001_resource_permissions.sql:115` →
 - `20260817000001_resource_permissions_core_v2.sql:78` →
 - `20260817000005_resource_permissions_security_fixes.sql:36` →
 - `20260817000006_custom_role_enforcement.sql:70`. The final version adds a custom-role branch (tenant-defined roles beyond the fixed enum).

Frontend surface: `src/features/settings/access/PermissionsMatrixTab.tsx` lets `school_admin` edit `user_permission_overrides` per staff member, grouped by resource domain.

Confirmed gap: `resource_open_actions` and `resource_default_role_grants` — the two lookup tables `has_resource_permission()` reads on every call — have **no enable row level security statement anywhere in the migration history**. Verified by grepping every migration file that references either table name (`20260817000001_resource_permissions_core_v2.sql`, `20260817000002...000004` domain migrations, `20260817000005_resource_permissions_security_fixes.sql`, `20260817000006_custom_role_enforcement.sql`) — none contains `enable row level security` or `force row level security` for these two tables, and no `revoke` statement removes their default PostgreSQL grants either. Both tables carry no `tenant_id` column (they're global reference data — the resource/action/role catalog is the same for every tenant), so this is not a cross-tenant *read* leak. It is a potential **write** exposure: if the `authenticated` Postgres role retains its default `INSERT/UPDATE/DELETE` grant on these tables (Supabase's default for new `public`-schema tables unless explicitly revoked — not yet confirmed against a live database), any authenticated user of any tenant could `insert` a row into `resource_default_role_grants` granting an arbitrary role a permission it shouldn't have, which `has_resource_permission()` would then honor for every tenant. **This needs a live check before Phase 3** (a simple anon-key `insert` attempt against `resource_default_role_grants` from an authenticated non-admin session) — flagging now as a preliminary Critical-severity candidate, not yet confirmed exploitable.

4. RLS coverage

- 362 `create policy` statements across the 81 migration files.
 - 32 pgTAP suites under `supabase/tests/rls/`, run via `supabase/tests/run.sh` against a real local Postgres (all 32 currently pass — last full run this session, unrelated feature work).
 - RLS is enabled with **both** `enable row level security` and `force row level security` on every table this review sampled directly (`tenants`, `users`, `students`, `guardians`, `teachers`, `attendance`, `exams`, `grades`, `fee_invoices`, `payments`, `employees`, and the batch-enabled generic-tenant-admin set: `academic_years`, `academic_terms`, `calendar_events`, `classes`, `subjects`, `fee_structures`, `salary_components`, `leave_types` — enabled via a `do $$ ... foreach t in array [...] ... execute format('alter table public.%I enable row level security', t) $$` loop, `20260713000005_rls_policies.sql:48-67` and `:245-263`). **Caveat:** a first-pass static grep for literal `alter table public.<name> enable row level security` under-reported RLS coverage by ~15 tables purely because of this dynamic-SQL loop pattern — every one of those was manually re-verified against the loop's array contents and confirmed enabled. The two genuinely missing (§3) were found only after that manual reconciliation, which is itself a note for future audits: **do not trust a static grep for RLS coverage in this codebase without accounting for the `foreach`-loop enablement pattern.**
 - Full per-table policy inventory (all 362) was not transcribed row-by-row in this pass given Phase 0's scope; recommend a scripted `pg_policies` dump against a live schema (once a non-production target exists) as the authoritative source rather than continuing to hand-audit migration SQL.
-

5. Edge functions (29, excluding `_shared`)

All share `supabase/functions/_shared/security.ts`. The common auth pattern, `requireRole(req, allowed)` (`_shared/security.ts:50-70`): 1. Extracts the `Authorization` header, builds a user-scoped Supabase client with the `anon` key (RLS applies). 2. Calls `auth.getUser()` — no user ⇒ 401. 3. Builds a **separate** `service_role` client purely to read the caller's own `role` / `tenant_id` from `public.users` (bypassing RLS only to read the caller's own row, not anyone else's). 4. Rejects with 403 if `profile.role` isn't in the function's `allowed` list.

This pattern was directly observed in-file for `record-fee-payment`, `process-fee-payment`, `generate-fee-invoices`, `enroll-finalize-billing`, `issue-fee-document`, and `telebirr-notify` earlier this session (all six were re-read and partially rewritten in this repo's own recent work) and matches the shared module every other function imports.

`verify_jwt = false` (public, no-auth-required) is deliberately set for a short allow-list of endpoints — confirmed via `supabase/config.toml` and the live deployed-function list (checked during this session's earlier deploy): `check-admission-status`, `check-login-attempt`, `submit-admission`, `sso-domain-lookup`, `telebirr-notify`, `upload-admission-document`, `verify-admission-bank-url`, `verify-id`. Every other function requires a valid JWT.

Edge functions never invoked by the client (candidate dead code, needs confirmation in Phase 6): not checked in this pass — requires cross-referencing every `supabase/functions/*` slug against every `callFunction(...)` / `fetch("../functions/v1/...")` call site in `src/`. Flagged as a Phase 6 task, not completed here.

6. Feature flags / plan gating

- `subscription_tiers` — `supabase/migrations/20260715000016_module_permission_matrix.sql:57`
 - `tenant_module_overrides` — same file, line 92 (per-tenant override of a tier's default module set)
 - `system_config` — `supabase/migrations/20260719000009_system_config.sql` (later extended in `20260806000001_security_settings.sql` and `20260817000006_custom_role_enforcement.sql`) — tenant-scoped and platform-scoped (`tenant_id is null`) key/value settings, e.g. `active_sms_provider`.
 - Enforcement of the module toggle is client-side only — see the confirmed gap in §1.
-

7. Phase 1 blocker: no non-production target

Hard Rule #1 of the source prompt: *"Never run against production. Confirm the Supabase project ref in the env file matches the one above before any write. If not, stop and ask."*

- `.env.local` exists but `VITE_SUPABASE_URL=https://localdev.supabase.co` — a placeholder, not a real, reachable Supabase project.
- `supabase/config.toml` defines a **local CLI dev stack** (`project_id = "school-saas"`, ports 54321/54322) — this requires `supabase start` (Docker), which is documented elsewhere in this repo's own tooling (`.claude/skills/verify/SKILL.md`) as **hanging on image pulls in this environment** and is not currently usable here.
- `supabase/seed.sql` is a genuinely well-designed staging fixture — two demo tenants, all 7 non-platform roles seeded across both for cross-tenant isolation testing — but it **refuses to run** unless `current_setting('app.environment') <> 'production'` (`supabase/seed.sql:6-11`), and there is no evidence in this repo of a second, separate Supabase *project* (a different project ref) where that setting is `'staging'` — every deploy and every live check performed in this session's prior work (migrations, Edge Functions, production verification) targeted the single project ref `livqynxlibmccaycseer`, which is production (confirmed repeatedly this session: real deploys, real customer tenant "Abadir Elementary School", real invoices).

Net effect: there is currently no safe place to run Phase 1's "onboard a new school as super-admin" step. Proceeding would mean either (a) running `supabase start` locally against a fresh, empty local Postgres — possible, but every RLS-shim/auth/storage emulation quirk this repo's own `verify` skill documents would need to be worked around, or (b) creating a second, genuinely separate staging Supabase project and pointing a real

`.env.local` at it. Neither has been done. **This audit stops here per Hard Rule #1 and the source prompt's own explicit instruction to stop after Phase 0.**

8. Proposed Phase 1 plan (not executed — for approval)

If a staging project is provided (or local `supabase start` is confirmed workable in this environment):

1. Fill in the `<<< >>>` fields the source prompt requires: `SUPABASE PROJECT` (staging ref), `SUPER-ADMIN LOGIN`, `TEST SCHOOL NAME` (e.g. `QA – Harar Model Secondary School`), `ACADEMIC YEAR` (EC), `RUN DATE`.
 2. Authenticate as the platform super-admin against the real `onboard-tenant` Edge Function (not raw SQL) — this repo has one; confirmed present at `supabase/functions/onboard-tenant/`.
 3. Narrate and capture the exact request/response for: tenant creation, first school-admin invite (via `invite-tenant-admin`, native Supabase Auth, no shadow user table), and whatever auto-seeding actually happens (grade levels, EC academic year/terms, fee categories, grading scale, storage folder) — read from the function's own code, not assumed.
 4. Record, per the source prompt's own per-step checklist: path exists? worked? side effects correct? audit-logged? idempotent on retry?
 5. Checkpoint again before Phase 2 (login-as-school-admin + isolation tests), per Hard Rule #6.
-

Executive summary (Phase 0 only)

System health (static-analysis impression only, not yet live-tested): the tenancy/RLS architecture is unusually disciplined for a project this size — a single pair of security-definer helper functions is the sole source of truth for tenant/role resolution, 362 policies exist across every sampled table with both `ENABLE` and `FORCE` row-level security, and 32 pgTAP suites exercise them. The resource-permission system shows real iteration toward correctness (four rewrites of `has_resource_permission()`, each narrower/safer than the last).

Top risks surfaced by Phase 0 alone (all need live confirmation, not final findings): 1. `resource_open_actions` / `resource_default_role_grants` have no RLS — potential unauthenticated-role privilege-grant write path. Preliminary Critical. §3. 2. **Module-plan gating is UI-only** — a disabled module's routes are not blocked at the data layer, only hidden from navigation. Preliminary Medium (mitigated by the fact that per-table tenant/role RLS still applies underneath — the exposure is "sees a feature they didn't pay for," not "sees another tenant's data"). §1. 3. **No non-production Supabase target exists** — every future write-based audit phase is blocked until one is provisioned. This is itself the most actionable finding: it blocks not just this audit but any future safe QA cycle. §7. 4. Two areas flagged as *needing a scripted, live-schema-based pass rather than continued hand-review*: the full 362-policy inventory (§4) and the edge-function-reachability check (§5). Static migration-file review has a demonstrated false-positive rate here (§4's RLS-loop caveat) that a direct `pg_policies` / route-call-site diff would eliminate.

What to fix first: #1 (RLS on the two resource-permission lookup tables) is a two-line migration (`enable` + `force row level security`), plus a `select`-only policy open to `authenticated` and a `write` policy

restricted to `service_role`) and should ship regardless of whether the rest of this audit proceeds — it doesn't require a staging project to fix, only to *verify*.